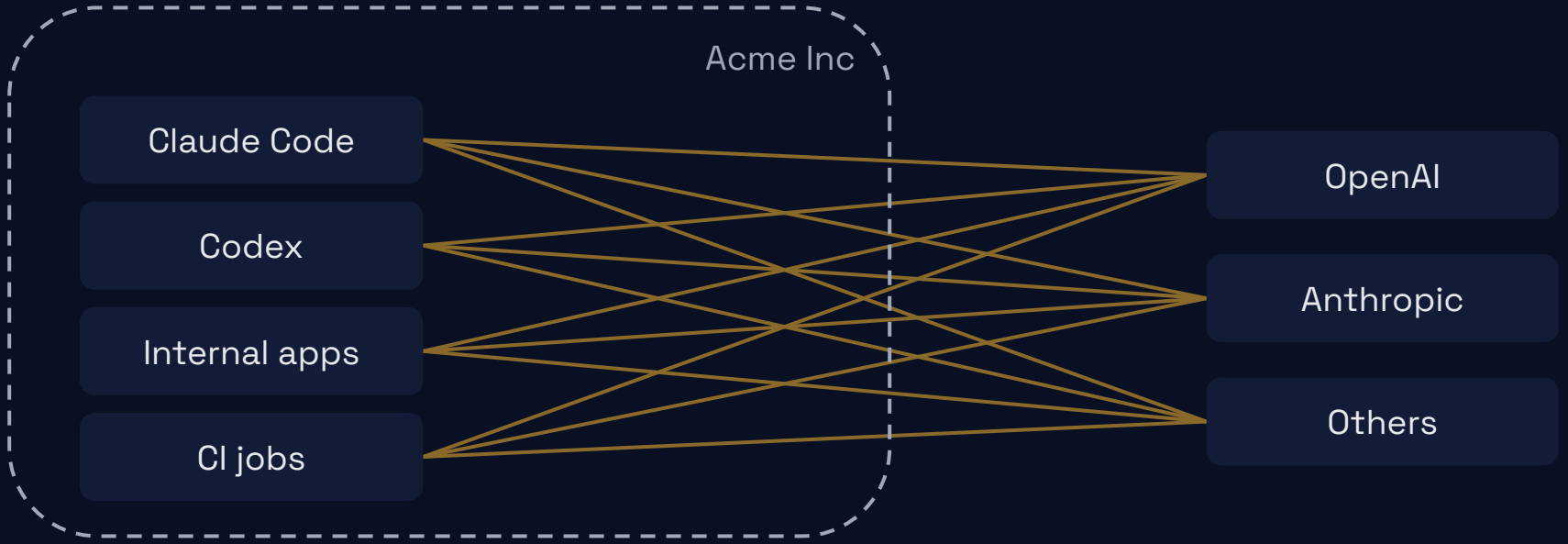
Keera Gateway

How Go keeps it fast



Every team does its own thing

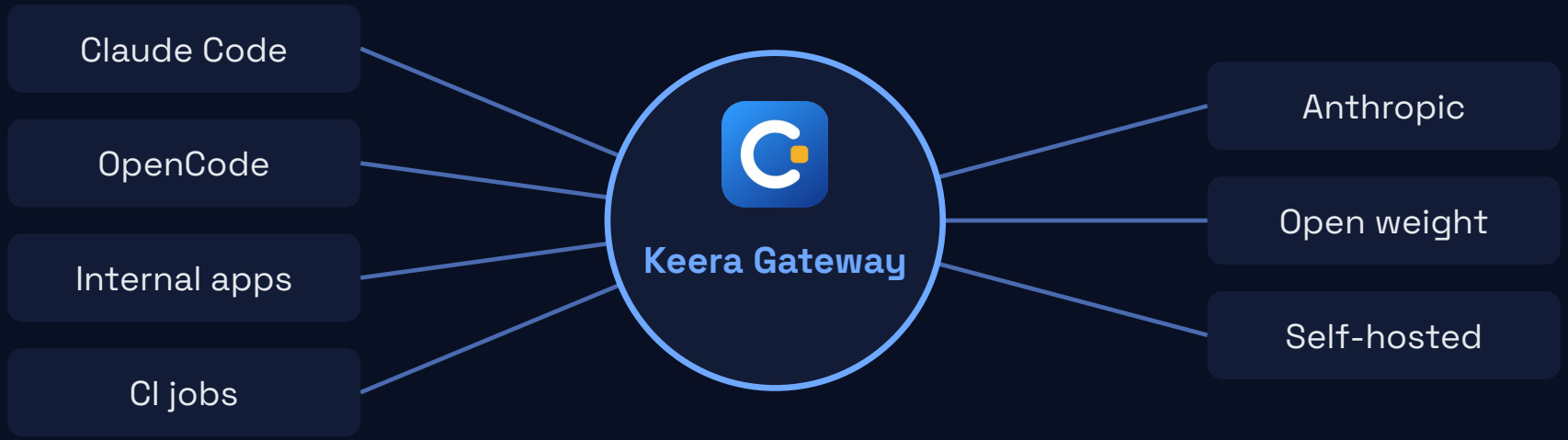


! No visibility

! No policy

! Unknown cost

! Data leaks


Claude Code
OpenCode
Internal apps
CI jobs
Keera Gateway
Anthropic
Open weight
Self-hosted

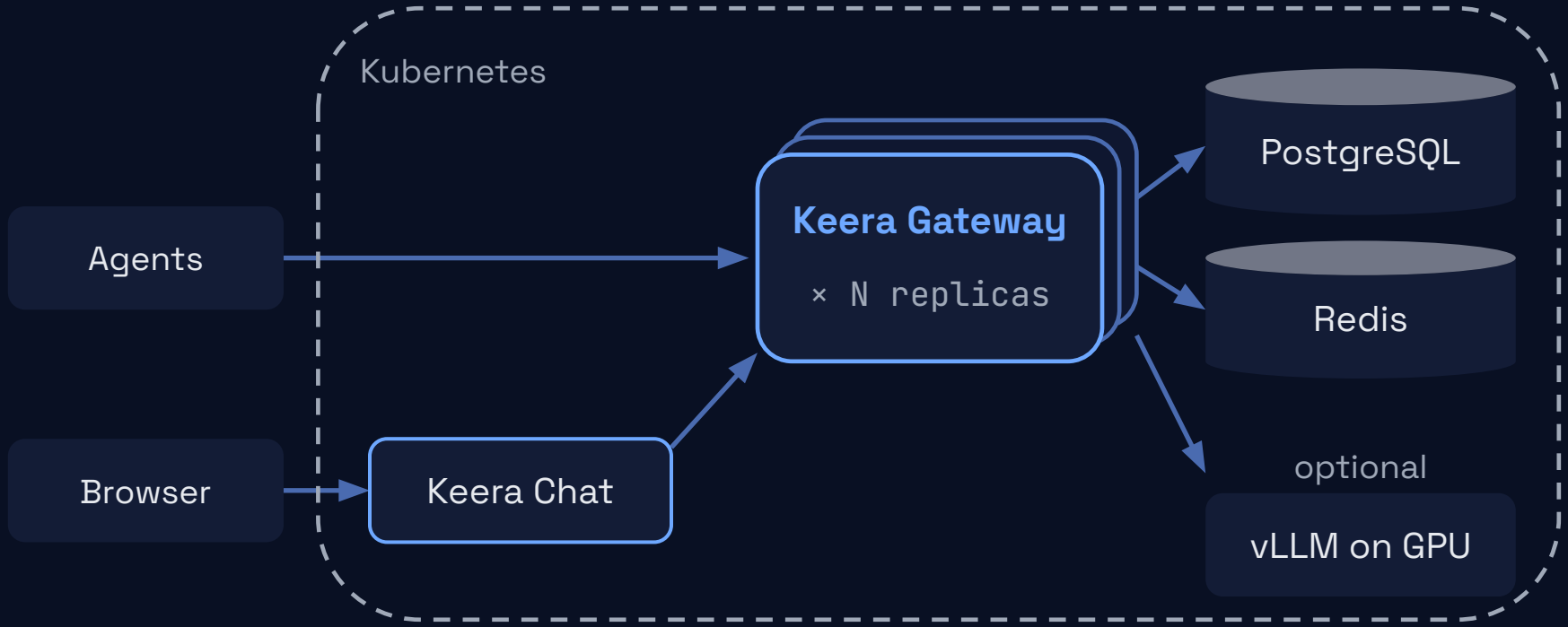

Keera Gateway
Dashboard
YOU
My access
Playground
ORGANISATION
Models
API keys
Routers
Filters
MCP
Projects
Usage
Fewer screens
ACTIVITY
Live map
Sessions
Requests
ADMINISTRATION
Users
Audit log
Billing
Collapse
Dashboard
Last 7 days
24h
7d
30d
REQUESTS
8
2 refused
TOKENS
1.2k
51 in · 1.2k out
COST
CHF 0.0011
FIRST TOKEN
982 ms
p95 13 s
Traffic
No failures
3
2
0
Oct 29
Oct 30
Oct 31
Nov 1
Nov 2
Nov 3
Nov 4
Nov 5
Projects by cost
All projects
default
CHF 0.001
team1
CHF 0.000058
Models by tokens
Usage report
gemma-4-31b-it
1.0k
gpt-6-luna
104
gemini-3-1-flash-lite
11
ministral-3b-2512
87

THE WORKLOAD

~600 KB

per request from a coding agent




Kubernetes
Agents
Browser
Keera Chat
Keera Gateway
× N replicas
PostgreSQL
Redis
optional
vLLM on GPU

One listener, three surfaces

/api	inference data plane
/control	control API
/	web panel

/metrics, /healthz ...

Why Go?



Very few dependencies

```
go 1.27.3

require (
    github.com/coreos/go-oidc/v3 v3.21.0
    github.com/go-jose/go-jose/v4 v4.1.8
    github.com/go-ldap/ldap/v3 v3.4.15
    github.com/jackc/pgx/v5 v5.12.0
    github.com/redis/go-redis/v9 v9.23.0
    golang.org/x/oauth2 v0.37.0
    gopkg.in/yaml.v3 v3.0.1
)
```

One static binary, built FROM scratch

```
FROM go:1.27
RUN CGO_ENABLED=0 go build \
      -mod=vendor -trimpath \
      -ldflags="-s -w" \
      -o /out/ ./cmd/keera-gateway


FROM scratch
COPY --from=build /out/keera-gateway /keera-gateway
USER 65534:65534
ENTRYPOINT ["/keera-gateway"]
```

The Keera CLI

```
keera login

keera model ls

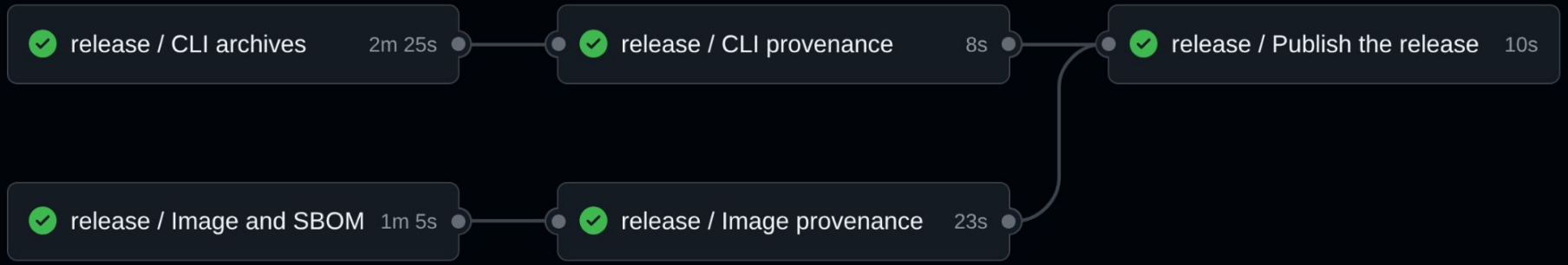
keera key create

keera router show

keera mcp add
```

SLSA build levels




release / CLI archives
2m 25s
release / CLI provenance
8s
release / Publish the release
10s
release / Image and SBOM
1m 5s
release / Image provenance
23s

Anyone can verify who built it

```
$ gh attestation verify \
    oci://ghcr.io/bespinian/keera-gateway:v0.13.0 \
    --repo bespinian/keera-gateway \
    --signer-workflow \
    bespinian/keera-gateway/.github/workflows/release-build.yml
```

Go Makes it Fast



How Fast?

	Keera	Bifrost	LiteLLM
Simple requests per second	1,000+ ✅	1,000+ ✅	200
Streamed chats per second	500+ ✅	100	below 100
Large agent requests per second	100+ ✅	100+ ✅	below 20
Added wait, typical	0.2–3 ms	0.2–2 ms	18 ms
Added wait, slowest 1%	≤ 7 ms	≤ 16 ms	84 ms
CPU for 100 chats/s	0.8 cores	1.6 cores	4 cores (overloaded)
Memory, 1,000 simple requests/s	33 MB	276 MB	3.4 GB
Memory, 100 agent requests/s	0.9 GB	3.9 GB	5 GB
When overloaded	never overloaded in the test	runs out of memory and crashes	waits of 30–60 s, then timeouts

How a Request Flows

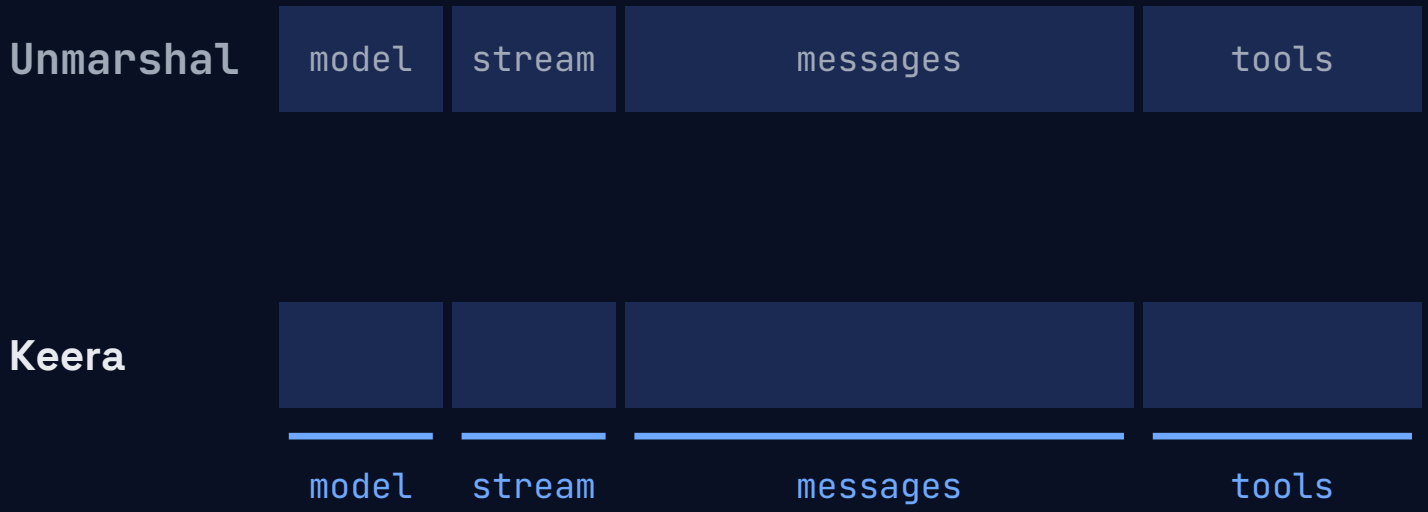


How a Request Flows



JSON

Decode Only the Top Level of JSON



JSON

Decode Only the Top Level of JSON

```
var decoders = sync.Pool{New: func() any {
    return jsontext.NewDecoder(new(bytes.Buffer))
}}
dec := decoders.Get().(*jsontext.Decoder)
defer decoders.Put(dec)
dec.Reset(bytes.NewBuffer(raw))

for dec.PeekKind() ≠ '}' {
    name, _ := dec.ReadToken()
    val, _ := dec.ReadValue()
    end := int(dec.InputOffset())
    // a window into raw, no copy
    fields[name.String()] = raw[end-len(val):end]
}
```

sync.Pool

jsontext

raw[a:b]

internal/gateway/body.go (simplified)

How a Request Flows



02

FILTERS

Only Scan What's New



FILTERS

Only Scan What's New

```
// Agents resend the whole conversation every turn,
// so remember the texts that were already clean.
var clean = map[[32]byte]bool{}

for _, text := range texts {
    key := sha256.Sum256([]byte(text))
    if clean[key] {
        continue // scanned last turn: skip
    }
    if !hasSecret(text) {
        clean[key] = true
    }
}
```

sha256

continue

hasSecret

internal/gateway/pattern.go (simplified)

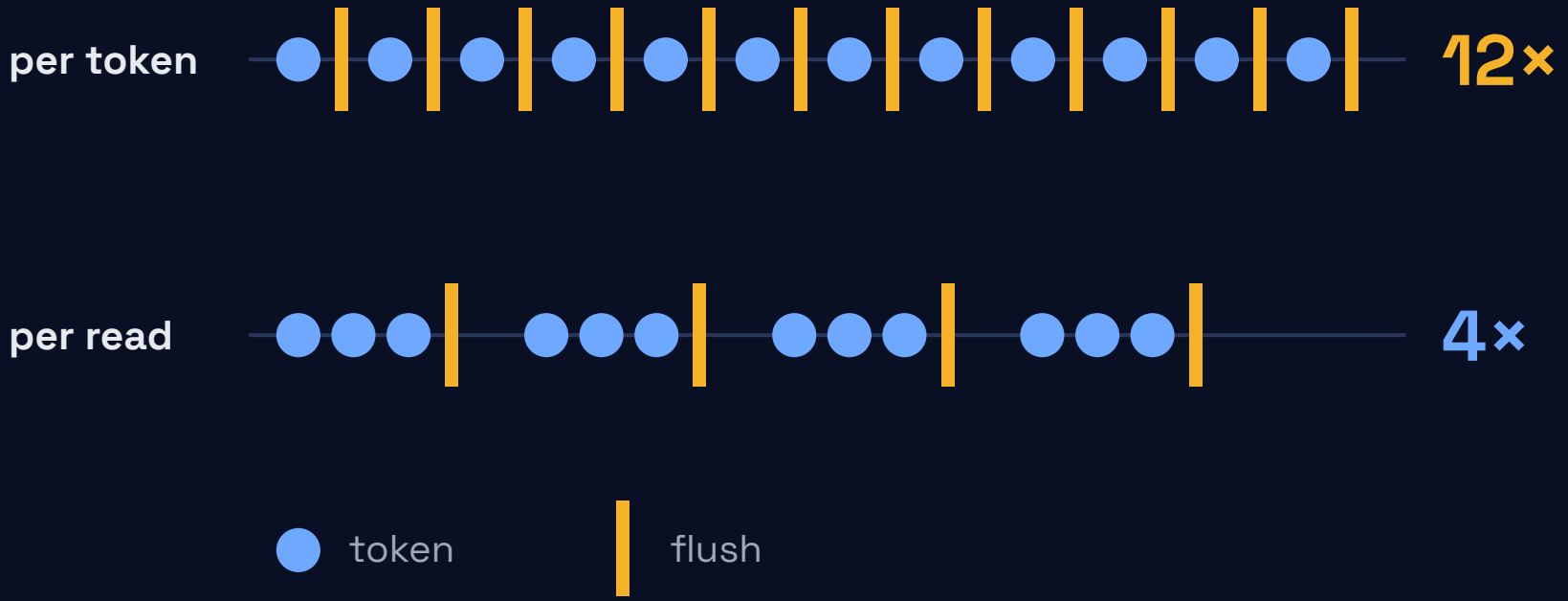
How a Request Flows



03

STREAMING

Dynamic Flushing in Streams



STREAMING

Dynamic Flushing in Streams

```
// Flush once before each read from the model,
// not once per token.
func (f *flushBeforeRead) Read(p []byte) (int, error) {
    f.rc.Flush()          // send what we have so far
    return f.src.Read(p) // then wait for the model
}
```

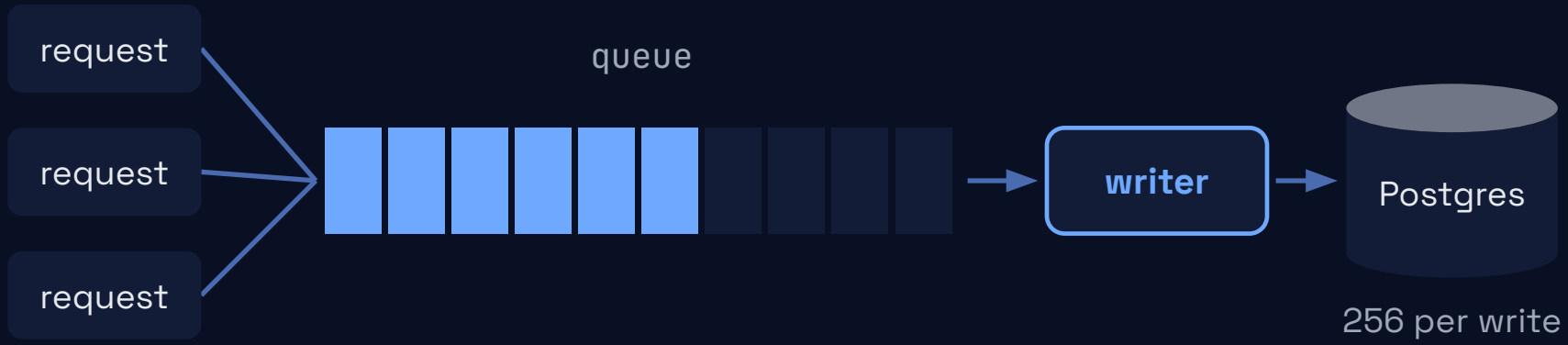
internal/gateway/sse.go (simplified)

How a Request Flows



USAGE LOG

Never Wait for the Database



USAGE LOG

The database never slows down inference

```go
func (r *Recorder) Record(e Event) {
    select {
    case r.queue ← e: // room in the queue: done
    case ←time.After(100 * time.Millisecond):
        r.dropped.Add(1) // DB too slow: drop it
    }
}

// started once, at boot:
go r.writeBatches() // 256 rows per INSERT
```

chan

time.After

go

internal/usage/recorder.go (simplified)

Tools

make
air
podman
gofmt -s
go fix

golangci-lint
goimports
gofumpt

Clödu

What’s not Go?

keera.ch

docs.keera.ch


Souveräne KI
«Die Schweizer Heldin
gegen die Schatten-KI»

Thank you

Questions?

keera.ch

github.com/bespinian/keera-gateway

```
go test -bench=. ./internal/gateway
```

